

LangGraph Debugging — Worked Example: Solution

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

- [Round 1: Basic](#)
- [Round 2: Intermediate](#)
- [Round 3: Real-world](#)
- [Round 4: Multi-agent](#)
- [Real cause vs. symptom fix](#)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Basic

Real cause: `route()` returns the string `"finish"`, but the mapping passed to `add_conditional_edges()` only knows about `"done"`, not `"finish"` — a plain naming mismatch between the routing function's return values and the mapping dictionary's keys. LangGraph has no way to guess what you meant, so it fails loudly, which is exactly the behavior Doc09's own Edge-cases exercise (`conditional_edge_mismatch`) is built to demonstrate.

The fix:

```
def route(state):
    if state["needs_tool"]:
        return "call_tool"
    return "done"

graph.add_conditional_edges("agent", route, {
    "call_tool": "tool_node",
    "done": END,
})
```

Make the routing function's return values match the mapping's keys exactly (and use LangGraph's actual `END` constant, not the string `"END"`, to close a branch out).

Test that would have caught it:

```
def test_route_return_values_match_the_conditional_edge_mapping():
    mapping_keys = {"call_tool", "done"}
    possible_return_values = {"call_tool", "done"}
    assert possible_return_values.issubset(mapping_keys)
```

More directly: actually invoking the compiled graph with an input that takes the "finish" branch, in a test, catches this immediately — this bug can hide for a while if only the "keep going" branch ever gets tested.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Intermediate

Real cause: by default, LangGraph merges a node's returned state update into the overall state field by field, and for a plain field (no reducer attached) the new value simply **replaces** the old one.

`calculate_node`'s `{"findings": {"calc": 42}}` doesn't merge with `search_node`'s earlier `{"findings": {"search": ...}}` — it overwrites the whole `findings` value outright, because nothing told LangGraph how to combine two updates to the same field.

The fix:

```
import operator
from typing import Annotated

def merge_findings(current, update):
    merged = dict(current)
    for key, value in update.items():
        merged[key] = value
    return merged

class AgentState(TypedDict):
    task: str
    findings: Annotated[dict, merge_findings]
```

Giving `findings` a reducer function tells LangGraph how to combine an existing value with a new update, instead of just overwriting — this is exactly Doc09's Core Concepts point about parallel branches needing a reducer to avoid the last update silently winning.

Test that would have caught it:

```
def test_findings_from_multiple_nodes_are_both_kept():
    state = {"task": "...", "findings": {}}
    state.update(search_node(state))
    state.update(calculate_node(state))
    assert "search" in state["findings"]
    assert "calc" in state["findings"]
```

A test that only ever runs one of the two nodes can't catch this — it has to run both, in sequence, against the same state, and check that both contributions survived.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Real-world

Real cause: `route()` only ever checks confidence against a fixed threshold — there's no counter tracking how many search rounds have already happened. For most questions, confidence climbs past 0.5 within a couple of rounds. For a specific kind of ambiguous question, results stay stuck in a "somewhat relevant but never quite enough" zone indefinitely, and with nothing else bounding the loop, it runs all the way to LangGraph's hard recursion limit before failing — the *symptom* (a recursion error) only fires 25 rounds after the *real cause* (a routing function with no round counter) first let the loop begin.

The fix:

```
class AgentState(TypedDict):
    task: str
    confidence: float
    search_rounds: int

def search_node(state):
    results = run_search(state["task"])
    new_confidence = score_confidence(results)
    return {
        "confidence": new_confidence,
        "search_rounds": state["search_rounds"] + 1,
    }

def route(state):
    if state["confidence"] >= 0.5:
        return "done"
    if state["search_rounds"] >= 4:
        return "give_up"
    return "search_again"
```

A real stopping rule needs two ways out, not one: "confident enough" and "tried enough times and still isn't" — `give_up` routes somewhere that returns a clear, honest "couldn't find a confident answer" result instead of quietly looping.

Test that would have caught it:

```
def test_search_loop_gives_up_after_a_fixed_number_of_rounds(fake_search_that_nev
    result = compiled_graph.invoke({"task": "a deliberately unanswerable question
    assert result["search_rounds"] <= 4
    assert result.get("gave_up") is True
```

This is Doc09's own Real-world exercise pattern: build a loop with no exit rule, on purpose, and run it with a low hard limit so it fails fast in testing instead of quietly running 25 rounds — the test above simulates exactly the "never quite confident enough" case that real ambiguous questions produce.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Multi-agent

Real cause: two separate bugs stacking on top of each other, each invisible on its own. First, `review_route` has the exact same missing-counter problem as Round 3 — no cap on revision rounds. Second, the specific reason the Reviewer keeps rejecting is that the "compliance detail" it wants was never actually written into `research_findings` in the first place (a Research-agent gap), so the Writer is being asked, every round, to include something it has no way to produce. Neither bug shows up testing the Writer/Reviewer pair alone with hand-written examples, because those examples always include a findings field the Writer *can* satisfy — the missing-data condition only occurs with real Research-agent output on certain real tasks.

The fix:

```
class SharedState(TypedDict):
    task: str
    research_findings: str
    draft: str
    review_passed: bool
    revision_count: int

def writer_node(state):
    draft = write_draft(state["task"], state["research_findings"])
    return Command(goto="reviewer", update={
        "draft": draft,
        "revision_count": state.get("revision_count", 0) + 1,
    })

def review_route(state):
    if state["review_passed"]:
        return "done"
    if state["revision_count"] >= 3:
        return "give_up"
    return "revise"

graph.add_conditional_edges("reviewer", review_route, {
    "done": END,
    "revise": "writer_agent",
    "give_up": "give_up_report",
})
```

The revision cap (matching Doc11's Build Task constraint directly) stops the infinite loop regardless of cause. Fixing *why* it kept looping still means going one layer further back — checking whether the Research agent's output can actually satisfy what the Reviewer is asking for, and if not, having the Reviewer's rejection reason point at what's missing from `research_findings`, not just "try again."

Test that would have caught it:

```
def test_revision_loop_gives_up_after_a_fixed_number_of_rounds(reviewer_that_never_satisfies):
    result = run_full_pipeline(sample_task_missing_a_compliance_detail)
    assert result["revision_count"] <= 3
    assert result.get("gave_up") is True
```

Running the *full* pipeline against a task deliberately missing something the Reviewer will want — not a hand-picked, always-satisfiable example — is what surfaces this; a 2-node Writer/Reviewer test with friendly fixtures structurally cannot reproduce a gap that only exists in real Research-agent output.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Real cause vs. symptom fix

A symptom fix would be raising the recursion limit whenever it's hit (Round 3) — the loop still never converges, it just gets to waste more tool calls before giving up — or having the Reviewer just approve everything after enough rejections (Round 4), which "fixes" the infinite loop by quietly shipping drafts that are missing the compliance detail nobody caught. Both leave the structural problem — no real stopping rule, and a Reviewer asking for something that was never available — fully in place. The fixes above instead add a real, honest exit condition at the exact layer the loop actually lives in (the conditional edge's own routing logic), which is Core Concepts' "checking layer by layer" point turned concrete: a graph that loops forever is a graph-structure bug, not something a bigger number or a more lenient reviewer can paper over.